CY Cergy Paris Université
UFR Sciences et Techniques
Département Informatique

Licence 2 Informatique

Génie Logiciel et Conception de Logiciels

RAPPORT DE PROJET

Simulateur d'Écosystème
Dynamique en Java

Conception et implémentation d'un simulateur d'évolution de biomes géré par événements

Feraoun Mohamed Amine

Anurajan Thenuxshan

Ouardani Nidhal

Encadrant :

Tianxiao LIU

Année universitaire :

2025 – 2026

Date de dépôt : Avril 2026

Table des matières

1 Introduction **6**

1.1 Problématique du projet 6

1.2 Enjeux de la conception logicielle 7

1.3 Objectifs du projet 7

1.4 Organisation du rapport 8

2 Spécification du simulateur **9**

2.1 Notions de base et contraintes du projet 9

2.2 Fonctionnalités attendues du projet 11

3 Conception et mise en œuvre **14**

3.1 Architecture globale du logiciel 14

3.2 Classes de données 16

3.3 Interface Homme-Machine graphique 17

3.4 Conception des traitements algorithmiques 21

4 Manuel d'installation et d'utilisation **25**

4.1 Prérequis et environnement technique 25

4.2 Récupération et compilation du projet 26

4.3 Lancement et utilisation de l'application 27

5 Déroulement du projet **30**

5.1 Organisation et répartition du travail 30

5.2 Phases de développement 31

5.3 Difficultés rencontrées et solutions adoptées 31

6 Conclusion et perspectives **33**

6.1 Bilan du travail réalisé 33

6.2 Perspectives d'amélioration 34

Table des figures

1 Logo du projet Environnement 6

2 Flux d'exécution d'un round de simulation 9

3 Exemple de grille de biomes générée par l'application 11

4 Architecture modulaire de l'application Environnement 14

5 Schéma conceptuel de l'interface de configuration initiale 18

6 Schéma conceptuel de l'interface de pilotage dynamique 19

7 Fenêtre principale de simulation avec ses trois zones fonctionnelles . . . 19

8 Fenêtre de tutoriel 20

9 Fenêtre d'édition : configuration de la carte et des probabilités d'événements 27

10 Fenêtre principale de simulation : grille, statistiques JFreeChart et contrôles 29

Liste des tableaux

1 Propriétés des événements météorologiques 12

2 Formations spatiales des événements météorologiques 13

3 Règles de transformation écologique et conditions d'activation 13

4 Bibliothèques prérequises pour le bon fonctionnement de l'application . 25

5 Contrôles de la simulation et leurs effets 28

6 Phases de développement et jalons du projet 31

Remerciements du groupe

Nous tenons à exprimer notre profonde gratitude envers toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet.

Nos remerciements s'adressent en premier lieu à notre professeur encadrant, M. Tianxiao LIU, pour ses précieux conseils, son suivi régulier et son accompagnement tout au long de ce projet. Ses orientations méthodologiques et techniques ont été déterminantes dans l'aboutissement de ce travail.

Nous remercions également l'UFR Sciences et Techniques de CY Cergy Paris Université pour les ressources mises à disposition et l'environnement propice à l'apprentissage qu'elle a su créer. Les enseignements dispensés au sein de la Licence 2 Informatique ont constitué le socle indispensable à la réalisation de ce simulateur.

Nos remerciements vont aussi à nos pairs pour les échanges constructifs et les réflexions partagées qui ont enrichi notre compréhension des concepts abordés.

1 Introduction

Le projet **Environnement** est une application de simulation écologique développée en Java 8. Une simulation écologique est un modèle informatique qui reproduit l'évolution d'un écosystème naturel au cours du temps, permettant d'étudier les interactions entre différents éléments et leur milieu. L'écosystème simulé est représenté par une grille où chaque cellule contient un *biome*, c'est-à-dire une région naturelle caractérisée par son climat et sa végétation. Sept types de biomes sont modélisés : Forêt, Désert, Mer, Montagne, Banquise, Ville et Village. Chaque biome possède des propriétés intrinsèques (température, humidité, pollution, purification) qui définissent son état environnemental à un instant donné.

FIGURE 1 – Logo du projet Environnement

La dimension temporelle de la simulation est structurée par des *rounds*. À chaque round, des événements météorologiques (Pluie, Vent, Pollution, Météore, entre autres) affectent les caractéristiques des biomes et peuvent déclencher des transformations automatiques selon des règles écologiques prédéfinies. Par exemple, une forêt soumise à une longue période de sécheresse peut se transformer en désert.

1.1 Problématique du projet

La problématique centrale consiste à concevoir un système de simulation permettant de gérer l'évolution dynamique d'un écosystème sur une grille bidimensionnelle, dans la tradition des automates cellulaires popularisés par Conway [Gar70]. Plusieurs questions ont guidé le développement : comment un biome réagit-il aux conditions environnementales changeantes telles que la température, l'humidité et la pollution ? Comment les événements climatiques propagent-ils leurs effets sur la carte ? Quand et selon quelles conditions un biome se transforme-t-il en un autre type ? Comment visualiser cette évolution de manière fluide et interactive sur une interface graphique ?

Ces questions convergent vers un objectif synthétique : créer une simulation dynamique et réaliste, accessible et pilotable via une interface graphique développée avec la bibliothèque Java Swing.

1.2 Enjeux de la conception logicielle

Ce projet présente plusieurs enjeux majeurs sur le plan de la conception logicielle. Les enjeux identifiés sont les suivants :

— **Extensibilité** : l'architecture doit permettre l'ajout de nouveaux types de biomes ou d'événements sans modifier le code existant, ce qui implique une séparation rigoureuse entre les données et les traitements.
— **Maintenabilité** : le code doit être modulaire et bien structuré pour faciliter les évolutions futures et la correction d'anomalies.
— **Travail collaboratif** : en équipe de trois développeurs, des conventions partagées et une conception claire sont indispensables pour garantir la cohérence du code et éviter les conflits.
— **Modélisation réaliste** : les règles de transformation écologique doivent refléter des phénomènes naturels plausibles tout en restant implémentables dans les contraintes du projet.

1.3 Objectifs du projet

L'objectif principal est de développer une application complète de simulation écologique reposant sur quatre fonctionnalités fondamentales :

— la création d'une carte configurable composée de sept types de biomes (Forêt, Désert, Mer, Montagne, Banquise, Ville, Village) ;
— l'apparition dynamique d'événements environnementaux (Pluie, VentFroid, VentChaud, Pollution, Purification, Météore et leurs variantes) ;
— la transformation automatique des biomes selon six règles écologiques : Inondation, Glaciation, Désertification, Forestation, Civilisation et Densification ;
— la visualisation en temps réel via une interface graphique interactive développée avec Java Swing.

Plusieurs fonctionnalités secondaires ont également été intégrées : une fenêtre de statistiques en temps réel avec des graphiques d'évolution générés par la bibliothèque JFreeChart, un mode édition interactif, une fenêtre de tutoriel permettant la prise en main rapide de l'application, ainsi qu'une option de contrôle de la vitesse de simulation.

1.4 Organisation du rapport

Ce rapport est structuré en six sections principales qui couvrent l'ensemble du cycle de vie du projet [Wik24]. La section 2 établit le référentiel fonctionnel : fonctionnalités attendues, terminologie, contraintes techniques et règles de comportement du simulateur. La section 3 détaille la conception logicielle et sa mise en œuvre : architecture modulaire, hiérarchie des classes, design patterns appliqués à notre code et traitements algorithmiques au cœur du moteur. Le manuel d'installation et d'utilisation, constituant un guide opérationnel destiné à tout utilisateur souhaitant prendre en main l'application, est proposé à la section 4. La section 5 retrace le déroulement concret du projet : organisation de l'équipe, phases de développement, outils employés et difficultés techniques surmontées. Enfin, la section 6 dresse le bilan du travail accompli et propose des perspectives d'amélioration pour les versions futures du simulateur.

2 Spécification du simulateur

Cette section détaille les fonctionnalités attendues du projet ainsi que les contraintes qui définissent son comportement. Elle constitue le référentiel fonctionnel sur lequel s'appuie la conception décrite dans la section 3.

2.1 Notions de base et contraintes du projet

Cette sous-section définit les fondements du modèle de simulation, établit le vocabulaire technique utilisé dans l'ensemble du rapport et recense les limitations connues de l'application.

Fonctionnement général du logiciel. Le fonctionnement repose sur un modèle de simulation par *rounds* successifs. À chaque round, le système exécute un cycle complet divisé en quatre étapes distinctes, comme l'illustre la figure 2.

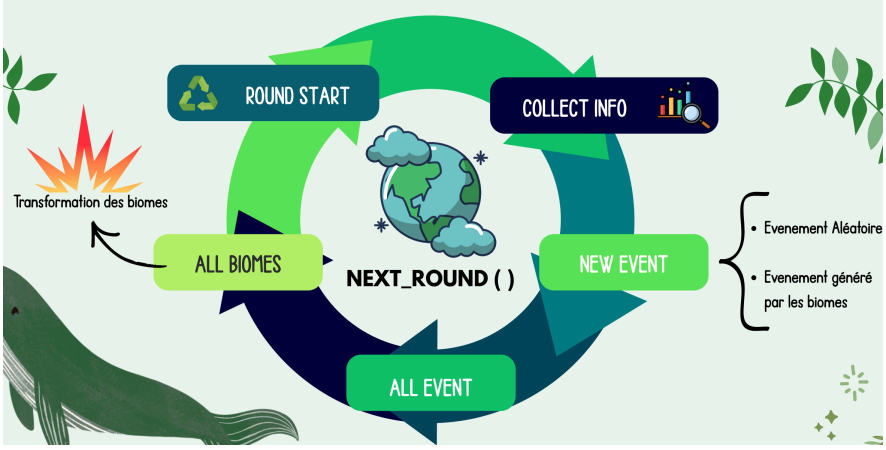

FIGURE 2 – Flux d'exécution d'un round de simulation

1. **Génération d'événements** : au début de chaque round, de nouveaux événements météorologiques apparaissent aléatoirement sur la carte selon des probabilités configurables.
2. **Déplacement des événements** : les événements mobiles se déplacent d'une case dans leur direction assignée ; ce déplacement peut être bloqué par les biomes Montagne.
3. **Application des impacts** : les événements modifient les propriétés numériques des biomes qu'ils occupent.

4. **Évaluation des transformations** : les règles de transformation vérifient les propriétés de chaque biome ; si les conditions sont satisfaites, le biome se transforme en un autre type.

La carte est représentée par une grille bidimensionnelle configurable dont chaque cellule contient exactement un biome. Chaque biome possède quatre propriétés numériques bornées entre 0 et 100 : température, humidité, pollution et purification.

Vocabulaire et terminologies de base. Les termes techniques récurrents dans ce rapport sont définis comme suit.

— **Biome** : unité de base de la carte représentant un type d'écosystème distinct (Forêt, Désert, Mer, Montagne, Banquise, Ville, Village).
— **Round** : cycle d'exécution de la simulation comprenant les quatre étapes de génération, déplacement, impact et transformation.
— **Événement** : phénomène météorologique affectant les propriétés numériques des biomes (Pluie, VentFroid, VentChaud, Pollution, etc.).
— **Transformation** : changement automatique d'un biome vers un autre type selon les règles écologiques définies.
— **Carte** : grille bidimensionnelle de blocs, chaque bloc contenant un biome identifié par ses coordonnées (x, y).
— **Propriétés** : les quatre caractéristiques numériques d'un biome, à savoir la température, l'humidité, la pollution et la purification.

Contraintes et limitations connues. L'application présente plusieurs contraintes et limitations identifiées lors de son développement.

1. **L'absence de persistance des données (Limitation technique).** L'état de l'écosystème est intégralement géré en mémoire vive durant l'exécution de la simulation. Le logiciel ne dispose pas de mécanisme de sérialisation pour sauvegarder l'état d'une simulation et la reprendre ultérieurement.
2. **Les contraintes liées à l'Interface Homme-Machine (IHM).** L'interface graphique possède une résolution fixée à 1920×1080 pixels. Le positionnement absolu des composants Swing impide un redimensionnement dynamique de la fenêtre. L'ergonomie du mode édition ne prend pas en charge la sélection de masse ni la duplication de zones. Une fois la simulation démarrée, la modification manuelle des cellules est désactivée pour garantir l'intégrité du cycle de simulation.

3. **L'absence de parallélisme (Limitation de performance).** Le moteur de simulation ne possède pas de parallélisme ; toutes les étapes d'un round s'exécutent séquentiellement dans un unique thread. Sur des cartes de grande taille ou avec un nombre d'entités élevé, cette approche séquentielle constitue un goulot d'étranglement car elle ne tire pas parti du multi-cœur du processeur.
4. **Les contraintes du modèle écologique (Limites conceptuelles).** Les quatre propriétés numériques d'un biome (température, humidité, pollution, purification) sont mathématiquement contraintes et bornées dans l'intervalle 0 à 100. Le biome Montagne constitue une exception : il est immuable et ses propriétés ne peuvent être modifiées par aucune règle ni aucun événement.

2.2 Fonctionnalités attendues du projet

Cette sous-section recense l'ensemble des fonctionnalités que l'application doit implémenter, organisées par domaine fonctionnel.

Gestion de la carte. La carte est représentée par une grille bidimensionnelle de cellules appelées *blocs*. Chaque bloc contient un biome et est identifié par ses coordonnées (x, y). Les dimensions de la grille sont configurables (par exemple 20×20 ou 30×30). La figure 3 montre un exemple de grille peuplée de biomes variés.

FIGURE 3 – Exemple de grille de biomes générée par l'application

La génération de la carte peut s'effectuer de manière aléatoire, les probabilités de chaque type de biome étant définies dans le fichier de configuration. Chaque bloc est accessible et modifiable via des méthodes dédiées. Le système permet de consulter ou de modifier l'état de chaque case de la grille. Les coordonnées sont validées avant tout accès pour garantir la cohérence des données.

Gestion des biomes. L'application modélise sept types de biomes, chacun représentant un écosystème distinct avec des propriétés environnementales spécifiques (température, humidité, pollution, purification). La Montagne constitue une exception notable : elle est immuable et ne peut être transformée par aucune règle. Les transformations entre biomes suivent des chaînes logiques reflétant des phénomènes écologiques plausibles ; par exemple, une Forêt soumise à la désertification devient un Désert, lequel peut, sous l'effet inverse de la forestation, redevenir une Forêt. Les valeurs initiales des propriétés de chaque biome et leur hiérarchie d'implémentation sont présentées dans la section 3.

Gestion des événements. L'application gère treize types d'événements divisés en deux catégories. Les événements statiques restent à une position fixe pendant leur durée de vie ; le Météore est le seul représentant de cette catégorie. Les événements mobiles se déplacent d'une case à chaque round dans une direction assignée. Le tableau 1 synthétise les propriétés de chaque événement.

Événement	**Catégorie**	**Durée**	**ΔTemp.**	**ΔHum.**	**ΔPoll.**	**ΔPurification**
Pluie	Mobile	4	0	+5	0	0
VentFroid	Mobile	4	−5	0	0	0
Pollution	Mobile	4	0	0	+10	0
Purification	Mobile	4	0	0	0	+5
Météore	Statique	20	+50	0	+20	0
Orage	Mobile	6	0	+10	0	0
Grêle	Mobile	3	−10	+5	0	0
Tornade	Mobile	8	0	+5	+5	0
PluieBénite	Mobile	5	0	+8	−5	+10
Zéphyr	Mobile	3	+5	0	0	0
Tonnerre	Mobile	2	0	+8	0	0
Smog	Mobile	6	0	0	+15	0
NuageToxique	Mobile	7	0	0	+20	0

Table 1 – Propriétés des événements météorologiques

Comme le montre le tableau 1, les événements mobiles exercent des impacts différenciés sur les quatre propriétés des biomes. Chaque événement mobile possède une durée de vie ; à son expiration, il est retiré de la carte. Lorsqu'un déplacement est bloqué par une Montagne, l'événement choisit une nouvelle direction aléatoire ; s'il n'en existe aucune, il reste en place.

Chaque événement est généré selon une formation spatiale définie, lui conférant une empreinte visuelle reconnaissable sur la grille. Le tableau 2 détaille la formation associée à chaque type d'événement.

Événement	**Formation**
Pluie, PluieAcide, Orage, PluieBénite	Carré 2×2
VentFroid, VentChaud	Ligne horizontale
Grêle	Ligne verticale
Tornade	Croix
Zéphyr	Diagonale
Tonnerre	Forme en T
Smog, NuageToxique	Cercle
Pollution, Purification, Météore	Unique

Table 2 – Formations spatiales des événements météorologiques

Système de transformation des biomes. Six règles de transformation écologique contrôlent l'évolution des biomes au fil des rounds. Le tableau 3 présente les conditions d'activation de chaque règle et le biome cible associé.

Règle	**Biome cible**	**Condition principale**
Inondation	Mer	Hum. ≥ 90, Temp. ≥ 5, Poll. ≤ 40
Glaciation	Banquise	Temp. ≤ 0, Hum. ≥ 20, Poll. ≤ 20
Désertification	Désert	Temp. ≥ 50 ou Hum. ≤ 10
Forestation	Forêt	Temp. $\in [10, 35]$, Hum. $\in [60, 90]$, Poll. ≤ 25
Civilisation	Village	Temp. $\in [15, 40]$, Hum. $\in [20, 80]$, Poll. < 10
Densification	Ville	Temp. $\in [18, 45]$, Hum. $\in [30, 70]$, Poll. $\in [10, 60]$

Table 3 – Règles de transformation écologique et conditions d'activation

Les conditions décrites dans le tableau 3 sont évaluées à chaque round pour chaque biome de la carte.

Interface graphique. L'interface graphique permet de visualiser et de contrôler la simulation en temps réel. La carte est affichée sous forme de grille colorée où chaque cellule représente un biome distinct. Les événements sont superposés à la grille sous forme de symboles animés. L'interface propose les contrôles suivants : Play pour démarrer la simulation, Pause pour la suspendre, Vitesse ×2 pour accélérer l'exécution, Bilan de fin pour consulter le récapitulatif, et Statistiques pour ouvrir la fenêtre d'analyse détaillée. Le mode édition permet de configurer manuellement la carte avant le lancement de la simulation ; une fois celle-ci démarrée, la modification manuelle des cellules est désactivée pour garantir l'intégrité du cycle de simulation. Une fenêtre de tutoriel est également disponible pour guider l'utilisateur lors de sa première utilisation.

3 Conception et mise en œuvre

Après avoir établi le référentiel fonctionnel dans la section 2, cette section décrit la conception logicielle et son implémentation. Elle explique comment les choix architecturaux répondent aux exigences du projet et comment le code a été structuré pour garantir l'extensibilité et la maintenabilité. Les composantes du système sont examinées successivement : l'architecture modulaire globale, la hiérarchie des classes de données, l'interface Homme-Machine et les traitements algorithmiques au cœur du moteur de simulation.

3.1 Architecture globale du logiciel

L'application est organisée en quatre modules distincts selon une architecture modulaire à responsabilités séparées. La figure 4 représente cette décomposition ainsi que les dépendances entre modules.

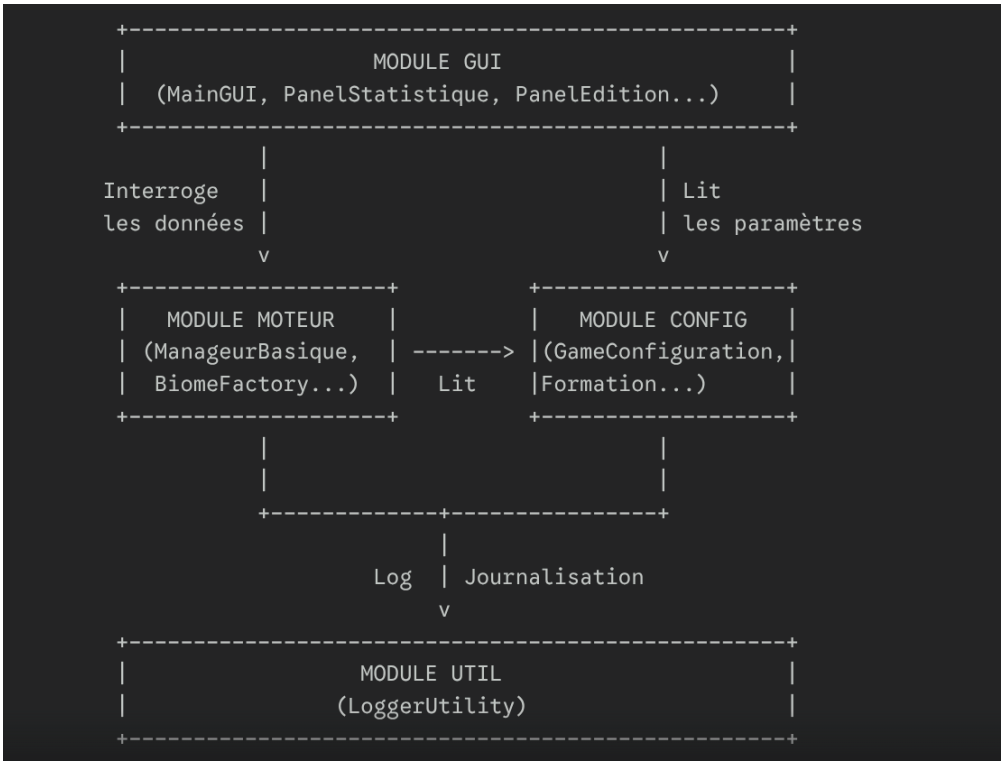

FIGURE 4 – Architecture modulaire de l'application Environnement

Les quatre modules identifiés dans la figure 4 sont les suivants.

— **Module `moteur`** : constitue le cœur de la simulation. Il contient la logique métier : gestion de la carte, des biomes, des événements et des règles de transformation. Ce module est indépendant de l'interface graphique et peut fonctionner sans interaction utilisateur.
— **Module `gui`** : gère l'interface utilisateur. Il contient les classes d'affichage (`MainGUI`, `MainDisplayer`, `PanelStatistique`, `PanelTemps`, `PanelEdition`, `TutorielPanel`) et la classe de rendu (`StrategiePeinture`). Ce module dépend du module `moteur` pour récupérer les données à afficher.
— **Module `config`** : centralise tous les paramètres : caractéristiques initiales des biomes, effets des événements, seuils de transformation et probabilités de génération.
— **Module `util`** : regroupe les utilitaires transversaux, dont le système de journalisation (`LoggerUtility`) et la configuration Log4j. Il est utilisé par tous les autres modules.

Cette séparation en modules garantit que toute modification d'un module n'impacte pas les autres, à condition de respecter les interfaces publiques définies. Elle facilite également les tests unitaires de chaque composant de manière isolée.

Positionnement MVC. L'architecture du projet s'articule autour du modèle MVC (Modèle-Vue-Contrôleur). Le Modèle (M) correspond au module `moteur`, qui contient les classes `ManageurBasique`, `Carte`, `Biome` et `Evenement`. La Vue (V) est assurée par le module `gui` via les classes `MainGUI` et `StrategiePeinture` qui gèrent le rendu graphique. Les Contrôleurs (C) sont les Listeners des boutons Swing, qui font le lien entre les interactions utilisateur et le moteur de simulation.

3.2 Classes de données

Cette sous-section décrit la conception des classes représentant les entités fondamentales de l'écosystème : biomes, événements et carte. Elle présente également les propriétés initiales des biomes, qui constituent un paramètre central du comportement de la simulation.

Hiérarchie des biomes. La classe abstraite `Biome` constitue la racine de la hiérarchie. Elle définit les attributs communs à tous les biomes (température, humidité, pollution, purification, position et bloc associé) ainsi que les méthodes d'accès et de modification de ces propriétés. Sept sous-classes héritent de `Biome` : `Foret`, `Desert`, `Mer`, `Montagne`, `Banquise`, `Ville` et `Village`. Chaque sous-classe initialise ses quatre propriétés avec les valeurs définies dans le module `config` (constantes de `ConfigurationBiome`). Ce couplage faible entre les valeurs de configuration et les classes concrètes permet de modifier les paramètres de simulation sans toucher à la logique métier.

Propriétés initiales des biomes. Le tableau **??** présente les valeurs initiales des quatre propriétés pour chaque type de biome, telles que définies dans la classe `ConfigurationBiome`. Ces valeurs servent de point de départ à chaque simulation et influencent directement la fréquence et la nature des transformations déclenchées.

Les propriétés listées dans le tableau **??** évoluent sous l'effet des événements climatiques et peuvent déclencher les transformations automatiques définies dans le tableau 3. La Montagne est la seule exception : ses propriétés ne peuvent être modifiées par aucune règle ni aucun événement.

Hiérarchie des événements. La classe abstraite `Evenement` représente la base de tous les phénomènes météorologiques. Elle contient les attributs communs : position sur la carte, direction de déplacement, durée de vie résiduelle et impacts sur les quatre propriétés des biomes. Deux catégories d'événements héritent de cette classe : les événements statiques (`Meteore`) qui restent à position fixe, et les événements mobiles (`Pluie`, `VentFroid`, `VentChaud`, `Pollution`, `Purification`, `Orage`, `Grele`, `Tornade`, `PluieBenite`, `Zephyr`, `Tonnerre`, `Smog`, `NuageToxique`) qui se déplacent d'une case à chaque round.

Gestion de la carte. La classe `Carte` gère la grille de simulation. Elle contient un tableau bidimensionnel d'objets `Bloc`. Chaque objet `Bloc` encapsule un biome et ses coordonnées (x, y). La classe `Carte` expose des méthodes pour accéder aux blocs via leurs coordonnées et pour valider la position avant tout accès, évitant ainsi les accès hors limites.

Le patron Fabrique [GHJV94, FRBS04] est mis en œuvre par les classes `BiomeFactory` et `EvenementFactory`. La `BiomeFactory` centralise la création de tous les biomes, permettant d'instancier un biome à partir de son type sans connaître la classe concrète. La `EvenementFactory` crée chaque événement selon une *formation spatiale* précise (unique, ligne, carré, T, L, diagonale, cercle, croix), définie dans le module `config`. Ce patron garantit que l'ajout d'un nouveau type de biome ou d'événement ne nécessite de modifier que la factory correspondante.

3.3 Interface Homme-Machine graphique

Cette sous-section décrit la conception de l'interface graphique réalisée avec la bibliothèque Java Swing [Cor23]. Le flux d'ouverture des fenêtres suit une logique séquentielle : l'utilisateur commence par le mode édition pour configurer la carte, puis lance la simulation, ce qui provoque l'affichage de la fenêtre principale.

Fenêtre d'édition. La fenêtre d'édition, implémentée par la classe `PanelEdition`, constitue le point d'entrée de l'application. Elle permet à l'utilisateur de configurer la carte avant le lancement de la simulation. La zone supérieure affiche la grille (`MainDisplayer`) où chaque cellule représente un biome. Un panneau latéral gauche (`PanelEdition`) donne accès aux sliders de configuration des probabilités d'événements, organisés par biome source (forêt, désert, mer, ville, village, banquise). La barre inférieure propose les boutons *Générer Carte* pour remplir la grille aléatoirement, *Lancer* pour démarrer la simulation, *Réinitialiser* et *Mettre à zéro* pour restaurer les valeurs par défaut, ainsi que *Modifier* pour changer manuellement un biome sélectionné sur la grille. Le mode édition est accessible uniquement avant le lancement de la simulation ; une fois celle-ci démarrée, la modification manuelle des cellules est désactivée.

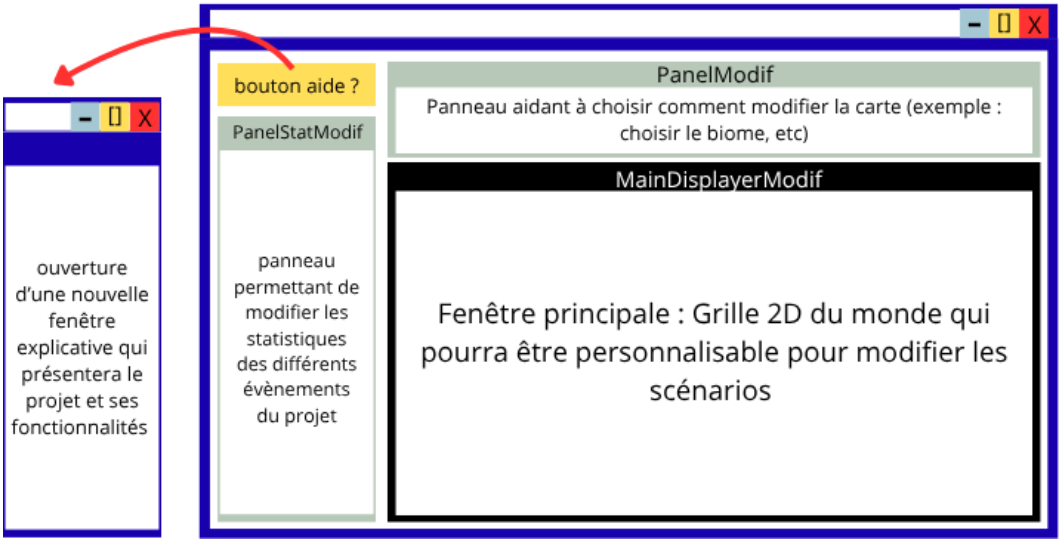

FIGURE 5 – Schéma conceptuel de l'interface de configuration initiale

Fenêtre de simulation. Lorsque l'utilisateur clique sur le bouton *Lancer*, la fenêtre d'édition se ferme et la fenêtre principale de simulation s'ouvre. Comme le montre la figure 7, elle est composée de trois zones fonctionnelles.

Les trois zones visibles dans la figure 7 sont les suivantes :

— la zone centrale (`MainDisplayer`) affiche la grille de simulation avec les biomes et les événements superposés ;
— le panneau latéral droit (`PanelStatistique`) présente les statistiques en temps réel (décompte de chaque type de biome) et les graphiques JFreeChart d'évolution des propriétés ;
— le panneau inférieur (`PanelTemps`) regroupe les contrôles de simulation (Play, Pause, Vitesse ×2, Bilan de fin).

La structure logique de ces contrôles de pilotage est illustrée dans la figure 6.

FIGURE 6 – Schéma conceptuel de l'interface de pilotage dynamique

FIGURE 7 – Fenêtre principale de simulation avec ses trois zones fonctionnelles

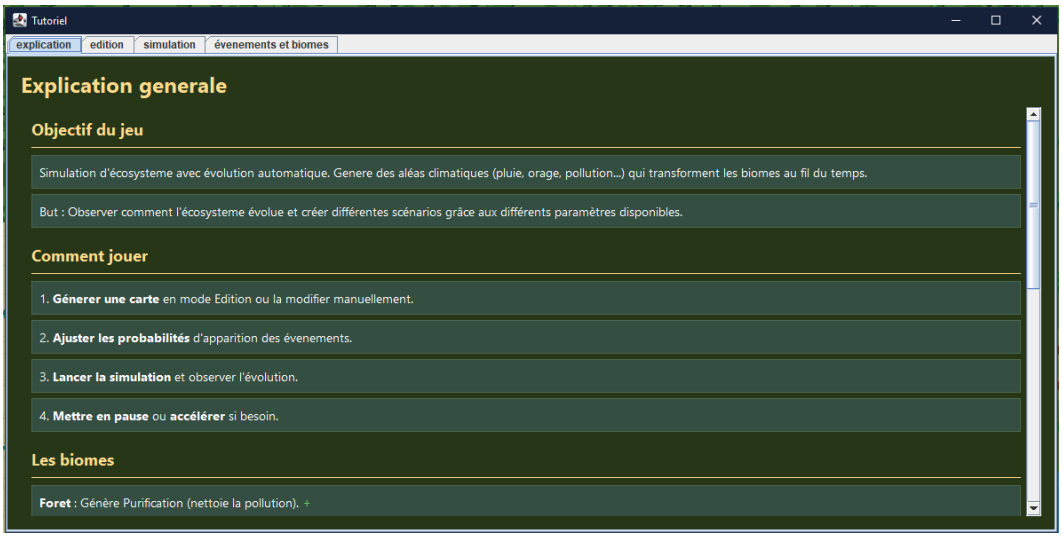

FIGURE 8 – Fenêtre de tutoriel

Fenêtre de tutoriel. Afin d'améliorer l'accessibilité de l'application, une fenêtre de tutoriel a été intégrée. Accessible depuis la fenêtre principale via un bouton dédié, elle est implémentée par la classe `TutorielPanel` et s'affiche dans une `JDialog` modale. Cette fenêtre présente les règles fondamentales de la simulation : le rôle de chaque type de biome, le comportement des événements et la logique des transformations écologiques. L'affichage modal garantit que l'utilisateur peut consulter ces informations contextuelles sans que la simulation en cours ne soit interrompue ni fermée.

Strategy de rendu. La classe `StrategiePeinture` centralise le rendu graphique de l'ensemble des éléments de la simulation. Chaque méthode `paintComponent` est spécialisée pour dessiner un type précis de biome ou d'événement, ce qui permet d'adapter l'apparence visuelle de chaque élément indépendamment du reste du code d'affichage. Cette spécialisation facilite l'ajout d'un nouveau rendu graphique sans modifier les classes de données ni le moteur de simulation.

3.4 Conception des traitements algorithmiques

Cette sous-section détaille la logique algorithmique et mathématique du module `moteur`. Les formules utilisées pour les transformations, la génération de la carte et la boucle principale de simulation y sont explicitées.

Formules mathématiques. Trois formules régissent le comportement quantitatif de la simulation.

La première formule concerne la **probabilité de génération d'un événement**. Pour chaque type d'événement, une probabilité est définie dans la classe `ConfigurationCreationEvenement`. Un événement est généré à chaque round si un nombre aléatoire tiré uniformément est inférieur à cette probabilité :

$$P(\text{génération}) = p_{\text{configurée}}. \tag{1}$$

La formule 1 permet de contrôler finement la fréquence d'apparition de chaque type d'événement.

L'équation 2 décrit le **calcul des impacts sur les propriétés**. Lorsqu'un événement affecte un biome, ses propriétés sont mises à jour par addition :

$$p_{\text{nouvelle}} = p_{\text{actuelle}} + \Delta p_{\text{événement}}. \tag{2}$$

L'équation 3 assure le **bornage des valeurs**. Toutes les propriétés sont contraintes dans l'intervalle $[0, 100]$ pour garantir la cohérence des données :

$$p_{\text{bornée}} = \max(0,\ \min(100,\ p))\,. \tag{3}$$

En Java, la formule 3 est implémentée via les méthodes `Math.max` et `Math.min`, appliquées systématiquement après chaque mise à jour de propriété.

Algorithme de la boucle de simulation. L'algorithme principal est implémenté dans la méthode `ManageurBasique.nextRound()`. Il exécute un cycle complet à chaque round selon la séquence décrite dans le listing 1.

```
ALGORITHME nextRound()
  GENERER nouveaux evenements aleatoires
  POUR CHAQUE evenement mobile FAIRE
    CHOISIR direction aleatoire
    DEPLACER evenement d'une case
    SI deplacement bloque (Montagne) ALORS
      CHOISIR nouvelle direction aleatoire
    FIN SI
    APPLIQUER impacts sur le biome cible
  FIN POUR CHAQUE
  POUR CHAQUE biome de la carte FAIRE
    EVALUER toutes les regles de transformation
    SI conditions d'une regle remplies ALORS
      TRANSFORMER biome selon la regle
    FIN SI
  FIN POUR CHAQUE
FIN ALGORITHME
```

Listing 1 – Pseudo-code de la méthode nextRound()

L'algorithme présenté dans le listing 1 garantit que les quatre étapes de la simulation sont exécutées dans un ordre déterministe à chaque round.

Algorithme de génération de la carte. La génération cohérente des biomes est implémentée dans `BiomeFactory.creerBiomesCoherents()`. Cet algorithme crée une distribution géographiquement plausible, avec des masses d'eau, des zones désertiques et des zones urbaines, comme le présente le listing 2.

```
ALGORITHME creerBiomesCoherents()
  GENERER masse d'eau (forme aleatoire : coin, haut ou milieu)
  POUR CHAQUE case de la grille FAIRE
    CALCULER distance a la masse d'eau par BFS
  FIN POUR
  POUR CHAQUE case proche de l'eau ET loin d'une zone desertique FAIRE
    AFFECTER biome Village
  FIN POUR
  POUR CHAQUE case loin de l'eau ET en zone desertique FAIRE
    AFFECTER biome Desert
  FIN POUR
  POUR CHAQUE case en zone urbaine FAIRE
    AFFECTER biome Ville
  FIN POUR
  RETOURNER carte avec biomes coherents
FIN ALGORITHME
```

Listing 2 – Pseudo-code de la génération cohérente de la carte

Les distances à la masse d'eau utilisées dans le listing 2 sont calculées par propagation BFS (*Breadth-First Search*), algorithme de parcours en largeur [CLRS09], pour déterminer les zones propices à chaque type de biome.

Règles de transformation écologique. Six règles implémentent l'interface `RegleTransformation`. Chaque règle évalue les propriétés d'un biome (température, humidité, pollution, purification) et retourne un nouveau biome si les conditions définies dans le tableau 3 sont satisfaites. Les règles implémentées sont : Inondation, Glaciation, Désertification, Forestation, Civilisation et Densification. Chaque règle encapsule ses conditions dans une méthode `appliquer(Biome b)` qui retourne le biome cible ou `null` si les conditions ne sont pas réunies ; le moteur applique simplement le premier résultat non nul.

Génération d'événements depuis les biomes. La classe `EvenementFactory` crée des événements selon des probabilités configurées dans `ConfigurationCreationEvenement`. Le patron Visiteur est incarné par la classe `GenerateurEvenementVisitor`, qui parcourt la carte et génère des événements supplémentaires depuis certains biomes (par exemple, un biome Mer génère périodiquement un événement Pluie selon son identité propre). Les probabilités de génération et les formations spatiales des événements sont entièrement définies dans le module `config`, ce qui permet de les modifier sans recompiler la logique métier.

Système de journalisation. Le système repose sur la bibliothèque Log4j [Fou23]. La classe utilitaire `LoggerUtility` fournit un accès centralisé au logger de l'application ; elle est conçue pour être utilisée statiquement depuis n'importe quel module. Les journaux sont émis aux niveaux `DEBUG` et `INFO`, et écrits simultanément vers la console et vers le fichier `src/logs/simulation.log`. Les actions journalisées incluent notamment le démarrage et la fin de chaque round, les déplacements d'événements, les transformations de biomes et les anomalies détectées lors de l'exécution.

Gestion des statistiques. L'affichage des statistiques en temps réel repose sur la bibliothèque JFreeChart [Gil20]. Les données collectées à chaque round (répartition des biomes, évolution des propriétés moyennes) sont injectées dans des `XYSeriesCollection` qui alimentent les graphiques du `PanelStatistique`. Ce mécanisme permet de visualiser l'évolution temporelle de l'écosystème avec une précision et une lisibilité supérieures aux représentations visuelles purement symboliques.

4 Manuel d'installation et d'utilisation

Cette section constitue le guide de référence destiné à toute personne souhaitant installer, compiler et utiliser l'application Environnement. Les instructions sont organisées de façon séquentielle : prérequis, procédure de compilation, lancement de l'application, puis guide d'utilisation de l'interface graphique.

4.1 Prérequis et environnement technique

Avant de procéder à l'exécution, il est nécessaire de vérifier que le Java Development Kit (JDK) 8 ainsi que les bibliothèques listées dans le tableau 4 sont présents sur la machine hôte. Un environnement de développement intégré est également requis pour compiler le projet ; Eclipse est recommandé car il a été utilisé lors du développement et garantit une compatibilité optimale avec la configuration du projet.

Bibliothèque	**Rôle**
jcommon	Utilitaires communs JFreeChart
hamcrest-core	Assertions pour les tests unitaires
jfreechart	Génération de graphiques 2D
jfreesvg	Export des graphiques au format SVG
junit	Framework de tests unitaires
log4j	Journalisation applicative
swtgraphics2d	Pont entre SWT et Graphics2D

Table 4 – Bibliothèques prérequises pour le bon fonctionnement de l'application

L'application est compatible avec les systèmes d'exploitation Windows 10 ou ultérieur, GNU/Linux (distributions Debian et dérivées) et macOS 12 ou ultérieur. Une résolution d'écran d'au moins 1920×1080 pixels est recommandée pour un affichage optimal de l'interface graphique.

4.2 Récupération et compilation du projet

La récupération du code source s'effectue via Git, puis la compilation est réalisée depuis Eclipse.

Récupération du code source. La méthode recommandée est d'utiliser GitHub Desktop, qui simplifie la gestion du dépôt. Après installation, le dépôt se clone via la procédure illustrée dans le listing 3 :

```
File > Clone repository > URL > coller l'URL du depot > Clone
```

Listing 3 – Clonage du dépôt via GitHub Desktop

Pour les utilisateurs ne souhaitant pas installer GitHub Desktop, il est possible de télécharger le projet directement depuis l'interface web de GitHub via la procédure illustrée dans le listing 4 :

```
Code > Download ZIP > extraire l'archive sur la machine locale
```

Listing 4 – Téléchargement du projet via l'interface web

Compilation avec Eclipse. Une fois le projet récupéré, il convient de l'importer dans Eclipse en suivant la procédure illustrée dans le listing 5 :

```
File > Import > Existing Projects into Workspace
  > selectionner le repertoire racine
```

Listing 5 – Import du projet dans Eclipse

Les bibliothèques listées dans le tableau 4 doivent ensuite être ajoutées au build path via la procédure illustrée dans le listing 6 :

```
Project > Properties > Java Build Path > Add External JARs
```

Listing 6 – Ajout des bibliothèques au build path

La compilation s'effectue ensuite automatiquement à chaque sauvegarde si l'option *Build Automatically* est activée dans le menu *Project*.

4.3 Lancement et utilisation de l'application

Une fois le projet compilé conformément à la section 4.2, l'application se lance depuis Eclipse en exécutant directement le fichier `Environnement.java` via la procédure illustrée dans le listing 7 :

```
Run > Run As > Java Application
```

Listing 7 – Lancement de l'application depuis Eclipse

Il est également possible de lancer l'application depuis un terminal si un fichier `.jar` exécutable a été exporté au préalable via la procédure illustrée dans le listing 8 :

```
java -jar /chemin/vers/Environnement.jar
```

Listing 8 – Lancement de l'application depuis un terminal

L'interface graphique s'ouvre automatiquement après quelques instants.

Étape 1 — Mode édition. Au lancement, la fenêtre d'édition s'affiche. Elle permet de configurer manuellement la carte avant de démarrer la simulation. La figure 9 présente l'aspect de cette fenêtre avec ses différentes zones de contrôle.

FIGURE 9 – Fenêtre d'édition : configuration de la carte et des probabilités d'événements

Comme on peut le constater sur la figure 9, les opérations disponibles dans cette fenêtre sont les suivantes :

— modifier les probabilités qu'un événement survienne via le menu de gauche ;
— réinitialiser ou remettre à zéro les probabilités des événements avec les boutons *Réinitialiser* ou *Mettre à zéro* ;
— cliquer sur un biome pour ensuite changer sa nature avec le bouton *Modifier* ;
— utiliser le bouton *Génération aléatoire* pour remplir automatiquement la carte avec une distribution pseudo-aléatoire de biomes.

Étape 2 — Lancement de la simulation. Le bouton *Lancer la simulation* ferme la fenêtre d'édition et ouvre la fenêtre principale de simulation. La simulation débute automatiquement dès l'ouverture de cette fenêtre.

Étape 3 — Contrôle de la simulation. Les contrôles disponibles dans le panneau inférieur (`PanelTemps`) permettent de piloter l'exécution. Le tableau 5 décrit chaque bouton et son effet.

Contrôle	**Effet**
Play	Démarre ou reprend la simulation après une pause
Pause	Suspend l'exécution à la fin du round en cours
Vitesse $\times 2$	Double la cadence d'enchaînement des rounds
Bilan de fin	Affiche un récapitulatif statistique de la simulation
Statistiques	Ouvre la fenêtre d'analyse détaillée en temps réel
Tutoriel	Ouvre la fenêtre d'aide et de présentation des règles

Table 5 – Contrôles de la simulation et leurs effets

Étape 4 — Lecture des statistiques. La figure 10 illustre l'interface principale lors d'une session en cours, avec ses trois zones fonctionnelles : la grille centrale de simulation, le panneau de statistiques et les contrôles de pilotage.

Tel qu'illustré dans la figure 10, le panneau latéral droit affiche les statistiques en temps réel ainsi que les graphiques d'évolution des propriétés générés par JFreeChart. En cliquant sur un biome de la grille centrale, ses propriétés détaillées (température, humidité, pollution, purification) s'affichent dans ce même panneau. La fenêtre *Statistiques*, accessible depuis le tableau 5, présente une analyse agrégée de l'ensemble de la simulation sous forme graphique.

FIGURE 10 – Fenêtre principale de simulation : grille, statistiques JFreeChart et contrôles

Consultation des journaux d'exécution. En cas d'anomalie ou à des fins d'analyse, les journaux de la session en cours sont accessibles dans le fichier `src/logs/simulation.log`. Chaque entrée de journal comporte un horodatage, un niveau de sévérité (`INFO` ou `DEBUG`) et un message descriptif de l'action enregistrée.

5 Déroulement du projet

Cette section retrace l'organisation du travail d'équipe et les étapes successives ayant conduit à l'application décrite dans la section 3. Elle présente la méthode de collaboration adoptée, la répartition des responsabilités, les outils de développement utilisés, et les principaux obstacles surmontés au cours du projet.

5.1 Organisation et répartition du travail

L'équipe est composée de trois développeurs : Feraoun Mohamed Amine, Anurajan Thenuxshan et Ouardani Nidhal. Afin de tirer parti de la modularité de l'architecture décrite à la section 3.1, la répartition des tâches a suivi un découpage fonctionnel cohérent avec les quatre modules du projet.

Répartition du travail. Feraoun Mohamed Amine a conçu l'architecture modulaire et le cœur du moteur de simulation (classes Carte, ManageurBasique, règles de transformation). Il a réalisé les tests unitaires et rédigé la première partie du rapport.

Ouardani Nidhal a mis en place le système de journalisation Log4j et a développé l'algorithme de génération cohérente de la carte et la logique de déplacement des événements mobiles. Il a contribué à la rédaction de la deuxième partie du rapport.

Anurajan Thenuxshan a implémenté le module de configuration et la fenêtre d'édition (PanelEdition), assurant l'interface de paramétrage des événements. Il a apporté un support technique sur les composantes graphiques et participé aux tests d'intégration.

Cette répartition n'a pas été strictement cloisonnée : des révisions croisées du code ont été systématiquement organisées pour garantir la cohérence globale du projet et mutualiser les connaissances techniques entre les membres de l'équipe.

Méthode de travail collaborative. Un dépôt Git commun [CS14] a servi de base au travail collaboratif. Chaque fonctionnalité a été développée sur une branche dédiée avant d'être fusionnée vers la branche principale après revue de code. Des réunions d'équipe hebdomadaires ont permis de synchroniser les avancées, d'identifier les dépendances entre modules et d'ajuster le planning.

5.2 Phases de développement

Le développement s'est articulé en quatre phases successives dont les jalons sont présentés dans le tableau 6.

Table 6 – Phases de développement et jalons du projet

Phase	Période	Objectifs et livrables
1	Semaines 1–2	Analyse des besoins, conception de l'architecture, mise en place de l'environnement de développement et du dépôt Git
2	Semaines 3–5	Implémentation des classes de données (biomes, événements, carte), module `config`, premières règles de transformation
3	Semaines 6–8	Développement du moteur de simulation (`ManageurBasique`), intégration des factories, réalisation de l'interface graphique
4	Semaines 9–10	Finalisation des différents concepts du projet (comme le mode édition), corrections des anomalies et bilan de fin ; rédaction du rapport

5.3 Difficultés rencontrées et solutions adoptées

Plusieurs difficultés techniques ont été rencontrées au cours du développement. Elles sont décrites dans la section 5.3 sous la forme d'un couple problème/solution afin d'en faciliter la compréhension.

Les statistiques. Problème. Une première version des statistiques reposait sur des bulles visuelles se remplissant proportionnellement à l'état du monde. Cette représentation s'est révélée imprécise et insuffisante pour suivre l'évolution quantitative de l'écosystème au fil des rounds.

Solution. La bibliothèque JFreeChart a été intégrée pour remplacer ce dispositif. Elle permet d'afficher des graphiques en courbes clairs alimentés en temps réel par les données de simulation, offrant une lecture précise et rigoureuse des propriétés des biomes au fil du temps.

Les transformations écologiques. Problème. La gestion dynamique des transformations de biomes s'est avérée complexe à implémenter : un premier prototype fondé sur des conditions imbriquées dans le `ManageurBasique` était difficile à maintenir et ne tenait pas compte des spécificités de chaque biome.

Solution. Un système de règles de transformation a été créé, matérialisé par l'interface `RegleTransformation`. Chaque règle est liée aux statistiques propres du biome concerné et encapsule sa logique de manière autonome. Le moteur se contente d'itérer sur la liste des règles, ce qui rend l'ensemble extensible et maintenable.

Dynamique des biomes. **Problème.** Dans une version initiale, la seule différence entre les biomes était d'ordre visuel (couleur et texture). Tous les biomes recevaient les mêmes types d'événements, ce qui rendait la simulation peu réaliste et les identités de biomes interchangeables.

Solution. Le patron Visiteur (`GenerateurEvenementVisitor`) a été mis en place pour que les événements générés dépendent du type de biome visité. Un biome Mer génère ainsi des événements de type Pluie, tandis qu'un biome Désert favorise des événements de Zéphyr ou de Tornade. Cette logique renforce l'identité unique de chaque biome.

Création des événements. **Problème.** L'apparition d'un événement sur une seule case était peu lisible et manquait de cohérence visuelle. Un événement de type Orage occupant une cellule unique avait le même impact visuel qu'une simple Pluie, rendant la lecture de la grille confuse.

Solution. Un système de formations spatiales a été implémenté directement dans l'`EvenementFactory` : chaque événement est désormais créé selon une formation géométrique définie (ligne, carré, croix, etc.), via des méthodes de création dédiées. Ce mécanisme apporte un boost visuel et logique significatif : chaque type d'événement possède une empreinte reconnaissable sur la grille, ce qui renforce la cohérence globale de l'interface.

6 Conclusion et perspectives

Ce rapport a présenté la conception et l'implémentation du projet Environnement, un simulateur d'écosystème dynamique développé en Java 8 pour le cours de Génie Logiciel et Conception. L'application constitue un système complet et fonctionnel, capable de simuler l'évolution de biomes soumis à des événements météorologiques variés et gouvernés par des règles de transformation écologique.

6.1 Bilan du travail réalisé

L'ensemble des fonctionnalités définies dans la section 2 a été implémenté. Les sept types de biomes sont modélisés avec leurs propriétés initiales et leur comportement propre. Les treize événements météorologiques, statiques et mobiles, sont générés selon des formations spatiales distinctes, déplacés et appliqués conformément aux spécifications. Les six règles de transformation écologique évaluent correctement les propriétés des biomes à chaque round et déclenchent les transformations attendues, telles que définies dans le tableau 3. Enfin, l'interface graphique permet de visualiser la simulation en temps réel et d'en contrôler le déroulement de manière interactive.

Sur le plan de la conception logicielle, l'architecture modulaire à quatre modules (`moteur`, `gui`, `config`, `util`) a démontré son efficacité : elle a permis un développement en parallèle, facilité les tests d'intégration et rendu les corrections d'anomalies plus rapides à localiser. Les design patterns mis en œuvre (Factory, Visiteur, programmation par interfaces) ont contribué à la maintenabilité et à l'extensibilité du code, deux critères fondamentaux identifiés dans la section 1.2. L'intégration de JFreeChart a apporté une valeur analytique, améliorant l'accessibilité des données pour l'utilisateur.

Le travail en équipe de trois développeurs, organisé autour du dépôt Git et de réunions de synchronisation hebdomadaires, s'est déroulé de manière productive. La répartition initiale des responsabilités par module, décrite dans la section 5.1, a limité les conflits de fusion tout en favorisant la cohésion globale du projet.

6.2 Perspectives d'amélioration

Plusieurs axes d'amélioration ont été identifiés à l'issue du développement.

Le premier axe concerne la **persistance des données**. L'implémentation d'un mécanisme de sérialisation au format JSON ou XML permettrait de sauvegarder et de recharger l'état complet d'une simulation. Cette fonctionnalité rendrait l'application utilisable dans un contexte pédagogique où les séances sont fragmentées.

Le deuxième axe vise l'**amélioration de l'interface graphique**. Une refonte vers une mise en page fluide et redimensionnable, recourant par exemple à la bibliothèque JavaFX, permettrait de lever la contrainte de résolution fixe identifiée dans la section 2.1. Cette refonte améliorerait également l'expérience utilisateur sur les écrans de petite taille.

Enfin, un quatrième axe envisage l'**enrichissement du modèle de simulation**. L'introduction de chaînes alimentaires entre espèces animales virtuelles peuplant les biomes, ou la modélisation de phénomènes climatiques globaux (cycles hydrologiques, effet de serre simplifié), accroîtrait considérablement le réalisme et la valeur pédagogique du simulateur.

Ces perspectives confirment que le projet Environnement, dans son état actuel, constitue une base solide et extensible sur laquelle des développements futurs peuvent s'appuyer.

Références

[CLRS09] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009.

[Cor23] Oracle Corporation. Creating a gui with swing - the java tutorials, 2023.

[CS14] S. Chacon and B. Straub. *Pro Git*. Apress, 2 edition, 2014.

[Fou23] Apache Software Foundation. Apache log4j 2 manual, 2023.

[FRBS04] E. Freeman, E. Robson, B. Bates, and K. Sierra. *Head First Design Patterns*. O'Reilly Media, 2004.

[Gar70] M. Gardner. Mathematical games - the fantastic combinations of john conway's new solitaire game "life". *Scientific American*, 1970.

[GHJV94] E. Gamma, R. Helm, R. Johnson, and J. Vlissides. *Design Patterns : Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994.

[Gil20] D. Gilbert. Jfreechart java chart library, 2020.

[Wik24] Wikibooks. Apprendre latex, 2024.